Systems Design | Reto 2: La caja negra

Objetivo y contrato

Construir un reemplazo HTTP que, para cada request y secuencia, iguale el status y el cuerpo del servicio vigente, incluidos errores y peculiaridades. El evaluador usa `Authorization: Bearer <token de equipo>`, llama `GET /health`, `POST /__reset` antes de cada prueba y envía tambien rutas desconocidas. Cada paso debe responder en menos de 5 s. La réplica no puede llamar al original ni a otro reemplazo.

Evidencia y límites

Tenemos 20 ejemplos de una secuencia de la mañana y experimentos propios hasta el ID 153; prueban casos concretos, no el comportamiento total. Se observaron reversiones de un depósito y una transferencia, rechazos E506/E507 y algunas combinaciones de ruta, método y credenciales. Siguen pendientes los límites generales, la precedencia completa, otros escenarios de reversión y combinaciones HTTP; no generalizamos casos puntuales. Las respuestas permiten inferir comportamiento observable, no conocer la arquitectura interna del original. La evaluación usa la versión vigente a las 20:00 Colombia (21:00 Bolivia).

Diseño propuesto

Este es el diseño de la réplica. En evaluación corre una instancia: adaptador HTTP/auth/rutas -> parser que conserva la entrada -> validación con reglas y precedencia inferidas (aún incompletas) -> handlers deterministas -> estado en memoria (cuentas, operaciones y contadores) -> serializador central de status/cuerpo. `/__reset` limpia el estado; la prueba compara status y cuerpo, no headers. Investigación, token personal, original, corpus y comparador permanecen en el banco local, fuera del runtime; el despliegue no tiene dependencia de red hacia el original.

Decisiones y trade-offs

Una instancia y estado en memoria simplifican secuencias y reset; se evita escalar horizontalmente porque repartiría el estado. Parser, validación y handlers separados permiten cambiar reglas con evidencia y localizar regresiones; se evita un switch monolítico. Respuestas deterministas sin LLM en runtime priorizan paridad, latencia y auditabilidad; no se necesita inferencia para reproducir un protocolo observable.

Cómo mediremos

Cerrar hipótesis pendientes con experimentos de una variable, conservar cada llamada al original en `experimentos.jsonl` y convertir hallazgos en pruebas diferenciales locales. Objetivo: responder dentro del presupuesto de 5 s por paso. Los 100 tests ocultos son 40 de un paso, 45 de 2-5 y 15 de 6-12; por eso priorizamos secuencias, estado tras errores/reset y paridad exacta de status+cuerpo. Repetir un canario tras el incidente anunciado y versionar cualquier cambio. Pausar ante `X-Frontier-Infra: rate-limit`: es infraestructura, no respuesta del servicio.

Riesgos y respuesta

Un cambio del original puede invalidar reglas cerca del cierre: comparar una batería centinela antes/después, registrar versión/fecha y diferencias, actualizar especificación y réplica y volver a validar. El orden de validación puede alterar muchas secuencias: probar conflictos de reglas y estado tras fallos. Mantener como supuesto todo lo no observado; una afirmación falsa perjudica la evaluación de especificación.

Fuentes: README y ejemplos oficiales del reto; experimentos propios en `output/experimentos.jsonl`. Diagrama complementario: arquitectura-reto-2. El PDF de entrega se genera en `output/pdf/systems-design-reto-2.pdf`.